Nama : Muhammad Nazril Ilham
Kelas : SISTER A

Buat dalam Format PDF, unggah di google drive sharing dengan mode `publik` kemudian link ditempel (paste) ke dalam form yang ada di assignment ini.

Esai 1

1. Jelaskan dengan singkat perbedaan antara proses dan thread.
2. Sebutkan dan jelaskan 3 keuntungan dari penggunaan multithreading.
3. Jelaskan skenario dimana multithreading dapat meningkatkan performa aplikasi (misalnya, server web).

Kasus 1:

1. Buat review mengenai literatur yang membahas perbedaan antara **model many-to-one**, **one-to-one**, dan **many-to-many** dalam multithreading.
2. Buatlah review contoh kasus dari dunia nyata di mana sistem menggunakan multithreading untuk meningkatkan efisiensi.

Kasus 2:

1. Buatlah program (pilih salah satu: java, python, rust, golang). Buat program sederhana yang menggunakan dua thread. Satu thread bertugas membaca data dari file, dan thread lain bertugas menampilkan data ke layar.
2. Buat modifikasi pada program sehingga proses pembacaan file dipecah menjadi beberapa thread yang bekerja secara bersamaan.

Analisis hasil eksekusi program multithreading. Tuangkan dalam tulisan bagaimana multithreading meningkatkan efisiensi aplikasi.

Jawab :

Esai 1

1. Proses vs thread
    a. Proses adalah sebuah program yang sedang dalam tahap eksekusi secara menyeluruh, yang mencakup berbagai elemen teknis seperti program counter, register, variabel, stack, hingga data section. Dalam pengoperasiannya, sebuah proses memerlukan pengalokasian memori serta sumber daya yang dilakukan secara mandiri dan terpisah dari proses lainnya.
    b. Thread merupakan suatu alur kontrol yang berada di dalam lingkup sebuah proses. Pada praktiknya, sebuah aplikasi diimplementasikan sebagai satu proses yang dapat menampung satu atau beberapa thread di dalamnya. Karena thread bekerja dengan cara berbagi memori dan sumber daya dengan thread lain dalam

proses yang sama, maka proses pembuatan serta pengaturan peralihan (context switch) dapat berlangsung secara lebih cepat dan ekonomis.

2. Keuntungan dari penggunaan multithreading sebagai berikut:
    a. Responsivitas karena penggunaan multithreading pada aplikasi dapat membuat program terus berjalan meski ada suatu bagian dari program yang diblok atay sedang menjalankan operasi yang berjalan lama. Contohnya VSCode, satu thread dapat melakukan proses pengindeksan kode atau analisis sintaks di latar belakang, sementara thread utama tetap sigap merespons ketikan dan navigasi pengguna. Contoh lainnya adalah pada aplikasi pengolah kata, di mana satu thread bertugas menangani input teks, sementara thread lain secara otomatis menjalankan fungsi pemeriksaan ejaan atau penyimpanan dokumen secara otomatis tanpa mengganggu aktivitas pengetikan.
    b. Berbagi sumber daya dikarenakan sebuah thread dapat berbagi memori dan sumber daya dengan thread lain yang berada dalam satu proses yang sama. Keuntungan dari mekanisme ini adalah efisiensi ruang, di mana sebuah aplikasi dapat menjalankan berbagai alur tugas secara bersamaan tanpa harus memisahkan lokasi memori untuk setiap aktivitas tersebut.
    c. Skalabilitas, di mana multithreading memungkinkan sebuah aplikasi untuk memaksimalkan potensi penggunaan CPU multi core. Tanpa multithreading, sistem dengan banyak core hanya akan menjalankan satu alur kontrol (single threading), yang mengakibatkan core lainnya menganggur dan penggunaan sumber daya menjadi tidak efisien. Dengan membagi tugas ke dalam beberapa thread, pekerjaan dapat dieksekusi secara paralel pada core yang berbeda sehingga performa sistem meningkat secara optimal. contohnya dalam implementasi algoritma Deep Learning, di mana pelatihan model yang melakukan ribuan perhitungan matriks dapat diselesaikan jauh lebih cepat karena diproses secara serentak, dibanding dengan menyelesaikan satu per satu berurutan.
3. Skenario yang dapat terjadi dimana multithreading meningkatkan performa aplikasi yaitu pada server web yang melayani ratusan atau ribuan klien secara bersamaan (concurrent), penggunaan multithreading dapat menjadi solusi untuk mengatasi beban pada server. Jika server cuma mengandalkan satu thread, server akan terbatas dalam melayani satu klien pada satu waktu, yang menyebabkan klien lain harus menunggu hingga proses untuk klien sebelumnya selesai. Multithreading dijadikan sebagai solusi dengan cara server memiliki satu thread utama yang berfungsi untuk mendengarkan permintaan masuk. Setiap kali ada permintaan baru, server akan membuat thread baru untuk melayani permintaan tersebut secara spesifik. Dengan menggunakan cara ini, server dapat terus merespons permintaan baru tanpa terhambat oleh proses yang sedang berjalan untuk klien lain, sehingga meningkatkan efisiensi dan kinerja server secara keseluruhan.

Kasus 1.

1. Buat review mengenai literatur yang membahas perbedaan antara model many-to-one, one-to-one, dan many-to-many dalam multithreading.
    a. Model Many-to-One

        Model many-to-one, yang sering dikaitkan dengan konsep "Green Threads" pada implementasi awal Java, memetakan banyak thread tingkat pengguna ke satu thread tingkat kernel tunggal. Dalam model ini, seluruh manajemen thread dilakukan oleh pustaka thread di ruang pengguna (user space), sehingga kernel tidak menyadari keberadaan thread-thread individu tersebut. Kernel hanya melihat satu entitas yang dapat dijadwalkan, yaitu proses itu sendiri.

        Manajemen thread di ruang pengguna memberikan keuntungan berupa efisiensi yang sangat tinggi dalam pembuatan thread, penghancuran thread, dan perpindahan konteks (context switching) karena semua operasi ini tidak memerlukan trap ke kernel atau perubahan mode dari user mode ke kernel mode. Namun, model ini memiliki kelemahan fatal dalam eksekusi paralel. Karena hanya ada satu thread kernel, seluruh proses akan terhenti (blocked) jika salah satu thread pengguna melakukan panggilan sistem yang memblokir, seperti operasi I/O. Selain itu, model ini tidak dapat memanfaatkan sistem multiprosesor karena kernel hanya dapat menjadwalkan satu thread kernel pada satu CPU pada satu waktu, terlepas dari berapa banyak core yang tersedia.

    b. Model One-to-One

        Model one-to-one memberikan korespondensi langsung di mana setiap thread tingkat pengguna dipetakan ke satu thread tingkat kernel yang unik. Model ini mengatasi masalah utama dari model many-to-one dengan memberikan visibilitas penuh bagi kernel terhadap setiap thread individu. Sebagian besar sistem operasi modern, termasuk varian Windows serta Linux, mengadopsi model ini sebagai standar utama mereka.

        Keuntungan utama dari model one-to-one adalah kemampuannya untuk menjalankan thread secara paralel pada multiprosesor. Selain itu, jika satu thread melakukan panggilan sistem yang memblokir, kernel dapat menjadwalkan thread lain dalam proses yang sama untuk terus dieksekusi. Kelemahan dari model ini adalah overhead yang lebih besar. Setiap kali aplikasi membuat thread pengguna, sistem operasi harus menciptakan thread kernel yang sesuai. Proses ini melibatkan alokasi sumber daya kernel yang signifikan, termasuk stack kernel yang biasanya berukuran tetap (sekitar 1MB), yang pada akhirnya membatasi jumlah thread yang dapat didukung oleh sistem tanpa menguras memori.

    c. Model Many-to-Many

        Model many-to-many adalah model yang mencoba menggabungkan elemen terbaik dari kedua model sebelumnya dengan mengalikan sejumlah thread tingkat pengguna ke jumlah thread tingkat kernel yang lebih sedikit atau sama. Dalam arsitektur ini, pengembang dapat membuat thread sebanyak yang diperlukan oleh logika aplikasi

tanpa khawatir membebani kernel secara berlebihan, sementara kernel hanya mengelola pool thread yang cukup untuk menjaga utilitas CPU tetap tinggi.

Variasi dari model ini adalah model dua tingkat (two-level model), yang memungkinkan hubungan many-to-many tetapi juga mengizinkan thread pengguna tertentu untuk "diikat" secara eksklusif ke satu thread kernel tertentu (1:1). Model many-to-many sangat efisien dalam menangani konkurensi tinggi, namun implementasinya sangat kompleks karena memerlukan koordinasi yang rumit antara pustaka thread di ruang pengguna dan penjadwal kernel. Jika satu thread kernel memblokir, mekanisme "Scheduler Activations" atau sinyal seperti SIGWAITING pada Solaris harus digunakan untuk memberitahu pustaka pengguna agar dapat memetakan kembali thread pengguna lain ke thread kernel yang masih aktif.

Aspek Perbandingan	**Many-to-One (N:1)**	**One-to-One (1:1)**	**Many-to-Many (M:N)**
Lokasi Manajemen	Ruang Pengguna (User Space)	Ruang Kernel (Kernel Space)	Hybrid (User & Kernel)
Paralelisme Hardware	Tidak didukung	Didukung penuh	Didukung penuh
Dampak Blocking I/O	Memblokir seluruh proses	Hanya memblokir satu thread	Dapat menjadwalkan thread lain
Overhead Pembuatan	Sangat rendah	Tinggi (perlu KLT)	Sedang
Pemanfaatan Core	Terbatas pada satu core	Efisien di banyak core	Sangat efisien dan fleksibel
Skalabilitas Jumlah Thread	Sangat tinggi	Terbatas oleh memori kernel	Sangat tinggi

Sumber :

1. https://mohitmishra786.github.io/exploring-os/src/day-07-thread-concept.html
2. https://www.cs.uic.edu/~jbell/CourseNotes/OperatingSystems/4_Threads.html
3. https://www.geeksforgeeks.org/operating-systems/multi-threading-models-in-process-management/
4. https://docs.oracle.com/cd/E19620-01/805-4031/6j3qv1oej/index.html
5. https://unstop.com/blog/multithreading-in-os
6. https://cs.brown.edu/research/thmon/thmon2.html
7. https://www.researchgate.net/publication/270217184_Thread_models_Semantics_Solaris_and_Linux_MN_to_11_thread_model

2. Buatlah review contoh kasus dari dunia nyata di mana sistem menggunakan multithreading untuk meningkatkan efisiensi.

   a. **Multithreading pada macOS: Grand Central Dispatch (GCD)**

   Apple menggunakan teknologi Grand Central Dispatch (GCD) atau libdispatch untuk mengoptimalkan paralelisme pada prosesor multi-core.

- Contoh Kasus & Efisiensi: Dalam aplikasi desktop macOS, GCD memungkinkan pengembang untuk memindahkan komputasi berat (seperti pengolahan gambar atau video) ke latar belakang agar tidak mengganggu responsivitas antarmuka pengguna (UI). Apple menyatakan bahwa GCD sangat efisien karena hanya membutuhkan sekitar 15 instruksi untuk memasukkan unit kerja (task) ke antrean, sementara pembuatan thread tradisional bisa memakan ratusan instruksi.
- Quality of Service (QoS): macOS menggunakan sistem prioritas berbasis QoS (seperti User Interactive untuk UI dan Background untuk pengindeksan file). Pada perangkat dengan Apple Silicon, GCD secara cerdas mendistribusikan tugas ke core performa (P-cores) atau core efisiensi (E-cores) berdasarkan kelas QoS tersebut untuk menghemat daya baterai.
- Performa: Penggunaan locking berbasis GCD terbukti sekitar 80% lebih cepat dibandingkan mekanisme sinkronisasi tradisional seperti @synchronized.

   b. **Multithreading pada Linux: Native POSIX Thread Library (NPTL)**

   Linux berevolusi dari implementasi awal yang kurang efisien (LinuxThreads) menjadi Native POSIX Thread Library (NPTL) yang menggunakan model pemetaan 1:1 yang sangat dioptimalkan.

- Contoh Kasus & Efisiensi: Database server seperti MySQL telah menggunakan multithreading selama bertahun-tahun di Linux untuk menangani ribuan permintaan konkuren secara efisien. NPTL memungkinkan pembuatan dan penghancuran thread menjadi sangat cepat; pengujian menunjukkan NPTL hampir 8 kali lebih cepat dibandingkan sistem lama dalam skenario pembuatan thread dalam jumlah banyak.
- Pemrosesan Data Masif: Dalam domain pengolahan citra digital (Digital Image Processing), operasi seperti konvolusi filter menunjukkan peningkatan performa yang signifikan di Linux saat diimplementasikan dengan pthreads karena kemampuan sistem untuk membagi beban kerja piksel secara independen ke berbagai core CPU.
- Skalabilitas: NPTL mengizinkan aplikasi untuk berskala hingga puluhan ribu thread dengan overhead yang tetap konstan, berbeda dengan sistem lama yang mengalami degradasi performa linier seiring bertambahnya thread.

   c. **Multithreading pada Windows: User-Mode Scheduling (UMS)**

   Windows secara standar menggunakan model 1:1 di mana setiap thread aplikasi dipetakan ke thread kernel, namun memperkenalkan User-Mode Scheduling (UMS) untuk kebutuhan efisiensi yang lebih tinggi.

- Contoh Kasus & Efisiensi: Aplikasi web browser modern pada Windows menggunakan pool thread latar belakang untuk menangani unduhan konten secara paralel sementara thread UI tetap responsif terhadap klik pengguna. UMS, yang diperkenalkan mulai Windows 7, memungkinkan aplikasi untuk melakukan perpindahan konteks (context

switching) di ruang pengguna tanpa melibatkan penjadwal kernel, yang secara signifikan mengurangi overhead.

- Studi Kasus Performa: Sebuah eksperimen pada sistem manajemen thread menunjukkan bahwa penjadwal tingkat pengguna pada Windows dapat memberikan peningkatan performa sebesar 93,5% dalam pembuatan thread dan 57,2% dalam kecepatan context switching dibandingkan dengan thread kernel tradisional.
- Manajemen Sumber Daya: Windows menggunakan blok data KTHREAD di dalam kernel untuk menyimpan informasi penjadwalan dan sinkronisasi bagi setiap thread, memastikan isolasi dan stabilitas sistem tetap terjaga meskipun aplikasi menjalankan banyak tugas sekaligus.

Aspek	**macOS (GCD)**	**Linux (NPTL)**	**Windows (UMS/Win32)**
Mekanisme Utama	Task-based queues (GCD)	1:1 Optimized KLT (NPTL)	1:1 KLT & Hybrid UMS
Keunggulan Utama	Overhead pembuatan sangat rendah (15 instruksi)	Kecepatan pembuatan thread 8x lebih cepat dari model lama	Kecepatan *context switching* naik hingga 57% dengan UMS
Fokus Efisiensi	Responsivitas UI & manajemen daya (QoS)	Skalabilitas tinggi untuk server database	Responsivitas aplikasi desktop & browser

Kasus 2:

1. Buatlah program (pilih salah satu: java, python, rust, golang). Buat program sederhana yang menggunakan dua thread. Satu thread bertugas membaca data dari file, dan thread lain bertugas menampilkan data ke layar.
2. Buat modifikasi pada program sehingga proses pembacaan file dipecah menjadi beberapa thread yang bekerja secara bersamaan.

Analisis hasil eksekusi program multithreading. Tuangkan dalam tulisan bagaimana multithreading meningkatkan efisiensi aplikasi.

Jawab:

Link github : https://github.com/Najer-jril/Sister/tugas_1

Hasil yang didapat

Hasil dari eksekusi program dengan dataset dari https://www.kaggle.com/datasets/ibrahimqasimi/pakistan-no-2018-2021 pada file year_2021.csv dengan detail 9 kolom dan 44.896.903 baris untuk masing masing kolom, menunjukkan bahwa implementasi multithreading pada bahasa Rust berhasil memangkas waktu pemrosesan secara drastis. Waktu eksekusi mencatat penurunan dari 2,1712 detik (Single Thread), menjadi 1,9279 detik (2 Thread: 1 Read, 1 Process), hingga mencapai waktu tercepat 0,9650 detik (Multi Thread dengan konfigurasi 10 threads).

Peningkatan performa yang signifikan ini dapat terjadi karena Rust mampu memaksimalkan seluruh arsitektur prosesor. Berbeda dengan bahasa tingkat tinggi lain, Rust tidak terhalang

oleh Global Interpreter Lock (GIL), sehingga seluruh thread dapat berjalan secara independen dan bersamaan di berbagai core fisik prosesor.
Hasil akhir bisa didapat karena adanya 2 faktor yaitu,

1. Pipelining I/O yang Efisien (Kasus 2 Thread) Pengujian 2 thread dengan waktu 1,9279 detik membuktikan bahwa konsep Overlapping I/O berjalan mulus. Saat satu thread bertugas menarik data dari SSD, thread kedua langsung menghitung barisnya tanpa harus saling tunggu. Komunikasi antar-thread di Rust sangat efisien, sehingga proses operan data ini tidak membebani sistem.
2. Paralelisme Murni Pembagian Data (Kasus 10 Thread) Lonjakan kecepatan tertinggi terjadi saat menggunakan 10 thread (waktu 0,9650 detik, atau lebih dari 2x lebih cepat dibanding Single Thread). Kecepatan ini dicapai karena file raksasa tersebut dipecah ukurannya secara matematis (berdasarkan byte). Ke-10 thread kemudian langsung "terjun" membaca bagiannya masing-masing secara serentak. Tugas yang awalnya murni menunggu kecepatan SSD kini berubah menjadi tugas komputasi paralel yang diproses berbarengan oleh prosesor.

Kesimpulan dari eksperimen ini membuktikan bahwa multithreading adalah kunci utama untuk meningkatkan efisiensi aplikasi saat menangani Big Data. Namun, performa yang sangat cepat yaitu di bawah 1 detik hanya bisa didapat jika bahasa pemrograman yang digunakan mampu mengelola konkurensi langsung di level sistem (system-level concurrency) tanpa terbebani oleh manajemen memori tambahan.

Source code:

```
use std::fs::{self, File};

use std::io::{BufRead, BufReader, Seek, SeekFrom};

use std::sync::mpsc::sync_channel;

use std::thread;

use std::time::Instant;



const FILE_PATH: &str = "../dataset/year_2021.csv";

const NUM_THREADS: u64 = 10;

const BATCH_SIZE: usize = 10000;

const QUEUE_SIZE: usize = 500;



fn single_thread() -> (f64, usize) {
```

```
    let start = Instant::now();
    let file = File::open(FILE_PATH).expect("gada filenya");
    let reader = BufReader::new(file);
    let mut line_count = 0;
    for _line in reader.lines() {
        line_count += 1;
    }

    (start.elapsed().as_secs_f64(), line_count)
}

fn two_threads() -> (f64, usize) {
    let start = Instant::now();
    let (tx, rx) = sync_channel::<Vec<String>>(QUEUE_SIZE);
    let producer = thread::spawn(move || {
        let file = File::open(FILE_PATH).unwrap();
        let reader = BufReader::new(file);
        let mut batch = Vec::with_capacity(BATCH_SIZE);

        for line in reader.lines() {
            if let Ok(l) = line {
                batch.push(l);
                if batch.len() >= BATCH_SIZE {
                    tx.send(batch).unwrap();
                    batch = Vec::with_capacity(BATCH_SIZE);
                }
```

```
            }
        }
        if !batch.is_empty() {
            tx.send(batch).unwrap();
        }
    });

    let consumer = thread::spawn(move || {
        let mut count = 0;
        while let Ok(batch) = rx.recv() {
            count += batch.len();
        }
        count
    });

    producer.join().unwrap();
    let total_lines = consumer.join().unwrap();

    (start.elapsed().as_secs_f64(), total_lines)
}
fn multi_threads() -> (f64, usize) {
    let start = Instant::now();
    let file_size = fs::metadata(FILE_PATH).unwrap().len();
    let chunk_size = file_size / NUM_THREADS;
    let (tx, rx) = sync_channel::<Vec<String>>(QUEUE_SIZE);
    let consumer = thread::spawn(move || {
```

```
        let mut count = 0;
        while let Ok(batch) = rx.recv() {
            count += batch.len();
        }
        count
    });

    let mut producers = vec![];
    for i in 0..NUM_THREADS {
        let tx_clone = tx.clone();
        let start_byte = i * chunk_size;
         let end_byte = if i == NUM_THREADS - 1 { file_size } else { (i + 1) *
chunk_size };

        let t = thread::spawn(move || {
            let mut file = File::open(FILE_PATH).unwrap();
            file.seek(SeekFrom::Start(start_byte)).unwrap();

            let mut current_pos = start_byte;
            let mut reader = BufReader::new(file);

            if start_byte != 0 {
                let mut discard = String::new();
                    let bytes_read = reader.read_line(&mut discard).unwrap() as
u64;
                current_pos += bytes_read;
            }
```

```
        let mut batch = Vec::with_capacity(BATCH_SIZE);
        while current_pos < end_byte {
            let mut line = String::new();
            let bytes_read = reader.read_line(&mut line).unwrap();
            if bytes_read == 0 { break; }

            current_pos += bytes_read as u64;
            batch.push(line);

            if batch.len() >= BATCH_SIZE {
                tx_clone.send(batch).unwrap();
                batch = Vec::with_capacity(BATCH_SIZE);
            }
        }
        if !batch.is_empty() {
            tx_clone.send(batch).unwrap();
        }
    });
    producers.push(t);
}
drop(tx);

for t in producers {
    t.join().unwrap();
}
```

```
    let total_lines = consumer.join().unwrap();
    (start.elapsed().as_secs_f64(), total_lines)
}
fn main() {
    let (d1, _c1) = single_thread();
    let (d2, _c2) = two_threads();
    let (d3, _c3) = multi_threads();

    println!("single, {:.4} s", d1);
    println!("1 read 1 process, {:.4} s", d2);
    println!("multi threads, {:.4} s", d3);
}
```